Формальные языки

Алфавит (Σ) — конечное непустое множество символов (чего-то)

Слово (строка) — конечная последовательность символов из Σ

Звезда (замыкание) Клини — множество всех конечных слов алфавита Σ:

$$\Sigma^* = \bigcup_{k=0}^{\infty} \Sigma^k$$

Формальный язык — конечное множетсво слов из алфавита $L \subseteq \Sigma^*$

Операции над языками

Теоретико-множественные:

- Объединение
- Дополнение (получается не язык)
- Мощность

Конкатенация:

- $L_1 \cdot L_2 = \{ab \mid a \in L_1, b \in L_2\}$
- $L^k = \underbrace{L \cdot L \cdot \ldots \cdot L}_{k \text{ раз}} = \{w_1 w_2 \ldots w_k \mid w_i \in L\}$
- $L^0 = \{\varepsilon\}$

Иерархия Хомского

Тип	**Класс**	**Машина**	**Пример**
3	Регулярные	Конечные автоматы	$\{a^n \mid n \geq 0\}$
2	Context-free	Автоматы с “магазином”	$\{a^n b^n \mid n \geq 0\}$
1	Context-Sensitive	Линейно ограниченный автомат	$\{a^n b^n c^n \mid n \geq 0\}$
0	Рекурсивно перечислимый	Машина Тьюринга	$*$

*$\{\langle M, w\rangle \mid M$ останавливается на $w\}$, то есть M либо завершается либо не завершается на w

Для типов справедливо, что $3 \subseteq 2 \subseteq 1 \subseteq 0$

Проблема разрешимости

Проблема разрешимости — вопрос, предполагающий ответ да/нет в зависимости от полученного слова

Примеры

1. $\mathrm{SAT} = \{\varphi \mid \varphi$ выполнима$\}$ (то есть φ как слово принадлежит языку SAT)
2. $\mathrm{VALID} = \{\varphi \mid \varphi$ валидна$\}$

Регулярные языки

Регулярные языки — класс REG, задается индуктивно:

- $R_0 = \{\varnothing, \{\varepsilon\}\} \cup \{a \mid a \in \Sigma\}$ — пустой язык и синглтон
- $R_{i+1} = R_i \cup \{A \cup B \mid A, B \in R_i\} \cup \{A \cdot B \mid A, B \in R_i\} \cup$
  $\cup \{A^* \mid A \in R_i\}$
- $\mathrm{REG} = \bigcup\limits_{k=0}^{\infty} R_k$

Теорема

REG *замкнуто относительно объединения, конкатенации и замыкания*

Доказательство

Ужасно тривиально, там в любом случае оба операнда принеадлежат каким-то (возможно разным) R_i. Отсюда следует вообще всё

QED

Регулярные выражения

Атомарные:

- $\varnothing$ — пустой язык
- ε — язык, содержащий только ε
- a — язык, содержащий все слова, содержащие ровно 1 букву из алфавита

Остальные (написано compound):

- $R_1 R_2$ — конкатенация R_1 и R_2
- $R_1 \mid R_2$ — объединение R_1 и R_2
- $R^* = RRR...$ — замыкание R
- (R) — ну R, только в скобках))))))))

Иногда языки по неназванным причинам называют греческими буквами: $A \leftrightarrow \alpha, B \leftrightarrow \beta$ и так далее

Конечные автоматы

Детерменированный конечный автомат (ДКА, DFA) — 5-кортеж $(Q, \Sigma, \delta, q_0, F)$, где

- Q — конечное множество состояний
- Σ — алфавит
- $\delta : Q \times \Sigma \to Q$ — функция переходов
- $q_0 \in Q$ — начальное состояние
- $F \subseteq Q$ — множество конечных состояний

Вычисление — процесс, релизуемый машиной $\mathcal{A}$ — конечная последовательность конфигураций (кадров/snapshots). Множество всех возможных конфигураций обозначается

$$\mathrm{SNAP} = Q \times \Sigma^*$$

Отношение достижимости — БО над конфигурациями:

$$\langle q, \alpha \rangle \vdash \langle r, \beta \rangle \iff \begin{cases} \alpha = c\beta \\ r = \delta(q, c) \end{cases}$$

То есть, $c_1 \vdash c_2$ значит, что c_2 достижимо из c_1 выполнением одного шага (то есть в слове α первой является именно символ, переводящий q в r, а остальная часть α совпадает с β)

Так же несложно заметить, что транзитивное замыкание $\vdash$ — отношение достижимости за любое количество шагов

Допустимое слово — слово $w \in \Sigma$ является допустимым для автомата $\mathcal{A}$ с начальным состоянием q_0, если в случае ввода слова w достижима конфигурация $\langle f, \varepsilon \rangle$, где $f \in F$ (состояние конечно), ε означает, что всё слово обработано

Распознаваемые слова — для конкретного автомата $\mathcal{A}$, множество слов, допустимых для него:

$$\mathcal{L}(\mathcal{A}) = \{w \in \Sigma^* \mid \langle q_0, w\rangle \vdash \ldots \vdash \langle f, \varepsilon\rangle, f \in F\}$$

Класс автоматных (automaton) языков — множество языков, распознаваемых каким-либо автоматом:

$$\mathrm{AUT} = \{X \mid \exists A : \mathcal{L}(\mathcal{A}) = X\}$$

Неопределенные конечные автоматы

Неопределенный конечный автомат (НКА, NFA) — 5-кортеж $(Q, \Sigma, \delta, q_0, F)$, где
- Q — конечное множество состояний
- Σ — алфавита
- $\delta : Q \times \Sigma \to \mathcal{P}(Q)$ — функция переходов
- $q_0 \in Q$ — начальное состояние
- $F \subseteq Q$ — множество конечных состояний

Определенная модель вычислений — модель вычислений, для которой каждый переход уникален ($\delta \to Q$)

Неопределенная модель вычислений — модель вычислений, в которой существуют переходы с возможным выбором ($\delta \to Q' \subseteq Q$)

Определения достижимости, допустимости и т.д. абсолютно аналогичны таким же для определенных автоматов. Дублировать не буду

ε-NFA — NFA, в котором модифицирована функция переходов:

$$\delta : (\Sigma \cup \{\varepsilon\}) \to \mathcal{P}(Q)$$

то есть, разрешены переходы между состояниями без символов вообще

Эпсилон-замыкание — для состояния q — множество состояний, достижимых из него с помощью ε-переходов:

$$E(q) = \{r \in Q \mid r \in \delta(q, \varepsilon)\}$$

Ну и несложно расширить определение эпсилон-замыкания на множества состояний:

$$E(P) = \bigcup_{q \in P} E(q)$$

Фан факт: $q \in E(q)$, т.к. можно попасть в q из q ничего не делая!

Теорема

Из любого ε-NFA *можно перейти к* NFA

Доказательство

1. Изображаем на графе транзитивное замыкание ε-переходов
2. Прокидываем "конечность" состояний в направлении, обратном напрвалениям ε-ребер (если ε-ребро заканчивается в конечном состоянии, начальное исходное этого ребра так же становится конечным, иначе ничего не происходит)
3. Подобно шагу 2 прокидываем в обратную сторону переходы по символам — если из конечной вершины есть переход $q \xrightarrow{\varepsilon} r \xrightarrow{\alpha} s$, такой путь заменяется на $q \xrightarrow{\alpha} s$
4. УДАЛЯЕМ ε-переходы

QED

Теорема

Из любого NFA *можно перейти к* DFA

Доказательство

Пусть в NFA все было примерно так:

- $Q_{\mathrm{N}} = \{q_1, ..., q_n\}$ (внимание! N и n никак не связаны)
- $\delta_{\mathrm{N}} : Q_{\mathrm{N}} \times \Sigma \to \mathcal{P}(Q_{\mathrm{N}})$

Тогда для DFA намутим вот так:

- $Q_{\mathrm{D}} = \mathcal{P}(Q_{\mathrm{N}})$
- $\delta_{\mathrm{D}} : Q_{\mathrm{D}} \times \Sigma \to Q_{\mathrm{D}}$
- Зададим δ_{D} как $\delta_{\mathrm{D}}(q, c) \mapsto \{r \mid \exists s \in q : r \in \delta_{\mathrm{N}}(q, c)\}$ (то есть переход $q \to r$ будет существовать только если из какого-то состояния внутри q можно прийти в состояние r)

- Ну и получается, что $F_{\mathrm{D}} = \{q \mid q \cap F_{\mathrm{N}} \neq \varnothing\}$ (то есть хотя бы одно состояние из q было конечным в F_{N})

Таким образом, из NFA получен очень даже валидный DFA. А этот алгоритм называется **методом подмножеств Рабина-Скотта**

QED

Теорема Клини

Теорема Клини

Язык регулярный тогда и только тогда, когда существует конечный автомат, распознающий этот язык:

$$\mathrm{REG} = \mathrm{AUT}$$

Доказательство

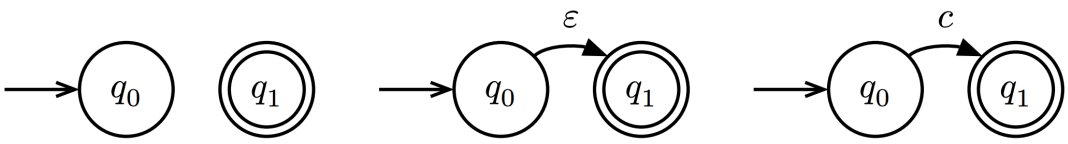
- ($\subseteq$) Построим ε-NFA с помощью построения Томпсона (см. ниже), распознающий язык, и превратим в DFA с помощью переходов, описанных выше
- ($\supseteq$) Из DFA построим регулярный язык с помощью алгоритма Клини (см. ниже)

QED

Построение Томпсона

Докажем, что $R_k \subseteq \mathrm{AUT}$ для всех k. Воспользуемся ММИ:

- База: $k = 0 \Longrightarrow R_0 = \{\varnothing, \{\varepsilon\}, \{c\}\ (c \in \Sigma)\}$. Для таких языков построим 3 отдельных автомата:



$L = \varnothing$ $\qquad$ $L = \{\varepsilon\}$ $\qquad$ $L = \{c\}$

- Шаг: пусть $R_k \subseteq \mathrm{AUT}$, покажем, что $R_{k+1} \subseteq \mathrm{AUT}$:
  $L, L_1, L_2 \in R_k \Longrightarrow L_1, L_2 \in \mathrm{AUT}$. Для R_{k+1} построим автомат:
  - Для $L_1 \cup L_2$ добавим новое исходное состояние и соединим ε-переходами с начальными состояниями L_1 и L_2
  - Для $L_1 \cdot L_2$ соединим ε-переходами конечные состояния L_1 с начальным состоянием L_2
  - Для L^* добавим новые конечное и начальное состояние с ε-переходами, формирующи цикл вокруг автомата (чтобы можно было и прочитать ε, и сколь угодно много раз L)

Алгоритм Клини

Обозначим R_{ij}^k регулярное выражение для слов, приводящих автомат из состояния q_i в состояние q_j, использующее при этом в качестве промежуточных состояний только $\{q_1, ..., q_k\}$. Тогда для $k = 0$ очевидно будет

$$R_{ij}^0 = \left\{a \mid \delta(q_i, a) = q_j\right\}$$

притом, при $i = j$ туда так же добавим и ε-строку

Чтобы из R_{ij}^{k-1} получить R_{ij}^k достаточно рассмотреть все слова, не содержащие q_k — это R_{ij}^{k-1} и все слова с переходами $q_i \to q_k$, затем сколь угодно много раз в q_k (именно из q_k) и переход $q_k \to q_j$, а это можно описать выражением

$$R_{ij}^k = \left(R_{ij}^{k-1}\right) \mid \left(R_{ik}^{k-1} \cdot \left(R_{kk}^{k-1}\right)^* \cdot R_{kj}^{k-1}\right)$$

ну и поскольку k убывает, когда-нибудь мы точно придем к уже определенным R_{ij}^0 при любых i и j.

Автоматы Мили и Мура

Оба автомата это 6-кортежи $(\Sigma, Q, \Delta, \delta\lambda, q_0)$, где

- Σ — входной алфавит
- Q — конечное множество состояний
- Δ — выходной алфавит
- $\delta : Q \times \Sigma \to Q$ — функция переходов
- λ — функция выхода (отличается в зависимости от автомата)
- q_0 — начальное состояние

Автомат Мили — автомат, описанный выше, в котором

$$\lambda : Q \times \Sigma \to \Delta$$

(значение возвращается **по переходу**, по ребру)

Автомат Мура — автомат, описанный выше, в котором

$$\lambda : Q \to \Delta$$

(значение возвращается по состоянию)

Невероятно, но факт — не так сложно делать переходы
Мили $\longleftrightarrow$ Мура

Пределы конечных автоматов

Слабая лемма о накачке

Пусть L регулярный, тогда существует некое $n \in \mathbb{N}$ (... насыщения), такая что для любое достаточно длинное слово $w \in L$ ($|w| \geq n$) можно представить в виде $w = xyz$, притом

- $y \neq \varepsilon$
- $(\forall i \geq 0)(xy^iz \in L)$

где (в автомате) y — цикл $q_k \rightsquigarrow q_k$

Доказательство

Следует из принципа Дирихле (приведем к DFA и там дальше все получается)

QED

Задача равенства — проверка на равенство данных слов x и y. По сути $x\#y$ это *вопрос* "равны ли x и y".

Аккуратно зададим язык

$$\text{EQUAL} = \{w\#w \mid w \in \{0,1\}^*\}$$

Теорема

EQUAL — *не регулярный язык*

Доказательство

Пойдем от противного — пусть EQUAL регулярный язык, а n — длина насыщения. Рассмотрим строку $0^n\#0^n \in \text{EQUAL}$. Попробуем насытить слово:

- Если $\# \in y$, то мы можем случайно удалить $\#$ ($i = 0$) или добавить несколько $\#$ ($i > 1$)
- Если $\# \notin y$, тогда мы можем "испортить" равенство строк, изменив количество нулей в левой (или правой) половине

Итак, лемма о насыщении не работает, а значит язык не мог быть регулярным

QED

Полная лемма о насыщении

Почти просто лемма о насыщении, но важное дополнение — зацикливание происходит уже в первые n шагов: $|xy| \leq n$

Доказательство

Воспользуемся слабой леммой о насыщении и принципом Дирихле

QED